
Self-Refined Distillation

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 On-policy distillation (OPD) has become a central post-training tool for large
2 language models (LLMs), providing dense per-token teacher supervision along the
3 student’s own rollouts. Its parameter-sharing variant, on-policy self-distillation
4 (OPSD), uses the same model as both student and teacher, with the teacher addition-
5 ally conditioned on privileged information such as the ground-truth answer. Despite
6 its promise, OPSD suffers from failure modes that existing remedies address only
7 through token-level loss interventions, such as truncation or reweighting. In this
8 work, we identify a common structural cause underlying several such failures,
9 which we call *prefix failure*. Under prefix failure, dense per-token supervision
10 induces a bimodal teacher mixture and fragmented gradients that token-level loss
11 interventions fail to address. This observation motivates moving beyond token-
12 level loss interventions toward trajectory-level output corrections. We propose
13 Self-Refined Distillation (SRD), a trajectory-level correction method that revises
14 the student’s rollout under the teacher guidance while within on-policy support.
15 By correcting problematic prefixes before distillation, SRD mitigates prefix failure
16 at its source. Moreover, SRD improves exploration by exposing the student to
17 alternative valid derivations under teacher guidance, even when original rollouts are
18 already correct. Across a wide range of benchmarks and base models at multiple
19 scales, SRD consistently outperforms prior baselines, improving single-attempt
20 accuracy and broadening reasoning coverage.

21 1 Introduction

22 On-policy distillation (OPD), which computes per-token teacher supervision along the student’s
23 own rollouts, has quickly secured a place in modern large language model (LLM) post-training (Gu
24 et al., 2024; Agarwal et al., 2024; Lu & Thinking Machines Lab, 2025). Recent industry releases
25 including Qwen3 (Yang et al., 2025), DeepSeek-v4 (DeepSeek-AI, 2026), MiMo-v2 (Xiao et al.,
26 2026), and GLM-5 (Zeng et al., 2026) all incorporate an OPD stage alongside supervised fine-tuning
27 (SFT) and reinforcement learning with verifiable rewards (RLVR). While OPD has achieved strong
28 performance, it typically relies on a distinct teacher model to supervise training. On-policy self-
29 distillation (OPSD) offers a lightweight alternative: it is a parameter-sharing variant of OPD in which
30 the teacher and student are the same model under different contexts. The student is conditioned only
31 on the problem statement, while the teacher is additionally conditioned on privileged information,
32 such as the ground-truth answer (Zhao et al., 2026; Hübötter et al., 2026; Shenfeld et al., 2026).

33 Despite these successes, recent studies show that vanilla OPD/OPSD recipes often fall short of their
34 promise, exhibiting failure modes that turn the supervision signal noisy or totally uninformative (Fu
35 et al., 2026; Xu et al., 2026b). Existing remedies address these issues at the token-loss level,
36 reweighting or clipping per-token contributions while leaving the sampled trajectory itself unchanged.
37 For example, Zhao et al. (2026) clip per-token losses above a fixed threshold to suppress destabilizing
38 high-KL tokens. Unfortunately, by acting only at the per-token loss level, these interventions fail
39 to mitigate *prefix failure*, an inherent limitation we formalize in this work. Prefix failure happens

when the student’s rollout takes a wrong reasoning path and almost no continuation of that prefix can reach the correct solution without backtracking or reflection. Under prefix failure, the per-token teacher distribution becomes a bimodal mixture between continuing the failed prefix and pivoting back toward the reference (Sec. 4.1); and even granting an ideal teacher, because the per-token KL is evaluated along the student’s *frozen rollout*, the resulting gradient is fragmented and its supervision pairs diverge from the correction path itself (Sec. 4.2). Recovery therefore requires a *trajectory-level* improvement; prior per-token approaches operate at the wrong scale, i.e., token-level, leaving the failed prefix structurally intact.

To mitigate the prefix failure, we propose Self-Refined Distillation (SRD), a simple yet effective trajectory-level refinement strategy that retains on-policy support while incorporating the reference solution as a guidance. We narrow our focus to the OPSD setting, which controls for confounds from teacher–student model mismatch in general OPD. Concretely, for each problem-solution pair, we first sample a raw on-policy rollout and then prompt the teacher model to produce a refined version of that rollout guided by the reference solutions. The refined trajectory is then used as supervision for subsequent OPSD updates with forward KL (Sec. 5). We empirically validate SRD on five competition-math benchmarks (AIME24/25, HMMT25, BeyondAIME, AMOBench) for Qwen3-4B and Qwen3-8B base models. SRD achieves state-of-the-art performance at both scales and delivers strong Pass@16 gains over the base (e.g., $\sim 50\%$ relative on AMOBench). Notably, on AMOBench SRD solves 9/23 (39%) of the questions the base model fails on all 16 sampling attempts, nearly doubling the best baseline (22%); these results confirm both single-attempt sharpening and reasoning-coverage expansion.

2 Related Work

On-Policy Distillation. On-policy distillation (OPD) for post-training LLMs traces back to classical knowledge distillation (Hinton et al., 2015) and replaces the fixed-corpus targets there with per-token teacher supervision computed along the student’s own rollouts. OPD couples with the on-policy sampling property and dense token-level learning signal by token-level KL loss (Gu et al., 2024; Agarwal et al., 2024; Song & Zheng, 2026; Yang et al., 2025; DeepSeek-AI, 2026; Xiao et al., 2026; Zeng et al., 2026). By contrast, RLVR offers only a sparse trajectory-level reward that scales poorly when most rollouts fail the verifier, whereas SFT relies on off-policy reference data and forfeits the on-policy structure that drives compute-efficient learning (Lu & Thinking Machines Lab, 2025).

On-policy Self-distillation. On-policy self-distillation (OPSD) is a special case of OPD that instantiates teacher and student from the same model under different privileged contexts, thereby removing the need for a separate teacher and enabling self-improvement without external supervision (Zhao et al., 2026; Hübotter et al., 2026; Shenfeld et al., 2026). The privileged context typically can be involved in reference solutions, feedback, knowledge and experiences, or other auxiliary information that is unavailable to the student (Shi et al., 2026; Ye et al., 2026; Penaloza et al., 2026; Wang et al., 2026; Stein et al., 2026). Operating within the OPD paradigm, OPSD inherits most of the same failure modes, e.g., the privileged supervision can collapse into vacuous guidance as training progresses. A similar concern appears in Zhao et al. (2026), who clip unusually large per-token losses to suppress unreliable learning signals and stabilize training.

Common Failure Mode and Fix. Recent studies report that those vanilla distillation methods often underperform in practice and exhibit a range of failure modes, including mode collapse, trajectory inflation, and supervision signals that vanish or even actively mislead the student and more (Fu et al., 2026; Xu et al., 2026b; Luo et al., 2026; Yang et al., 2026a; Kim et al., 2026; Li et al., 2026a; Xu et al., 2026a; Song & Zheng, 2026). Most of these failure modes are addressed through token-level dense-KL loss interventions. For example, Fu et al. (2026) find that the teacher distribution under OPD can be dominated by a small number of high-loss tokens, and propose top- K truncation to restrict supervision to high-confidence tokens. Xu et al. (2026b) upweights informative tokens, e.g., tokens with low student entropy but high teacher–student divergence, to achieve better results.

While the per-token interventions above may look contradictory (clipping high-loss tokens versus amplifying them), they share the goal of selecting informative learning signals, with the specific choice dictated by the divergence choice (Sec. 4.1). These interventions are motivated by empirically observation, yet the empirical failures in OPSD may partly be attributable to *prefix failure* (Sec. 4).

3 Preliminaries

On-policy Distillation. Knowledge distillation trains a student model π_S to match the output distribution of a teacher π_T , beyond the hard reference label (Hinton et al., 2015). In on-policy distillation (OPD) of LLMs, supervision is computed on trajectories sampled from the current student rather than on fixed expert prefixes (Gu et al., 2024; Agarwal et al., 2024; Lu & Thinking Machines Lab, 2025). Given a prompt $x \sim \mathcal{D}$ from the training dataset, the student samples an autoregressive rollout $y \sim \pi_\theta(\cdot | x)$. The teacher $\pi_T(\cdot | x, y_{<t})$ is then evaluated on the student-visited prefixes $y_{<t}$, producing a dense token-level learning signal. A representative OPD objective minimizes the per-token reverse KL divergence between student and teacher,

$$\mathcal{L}(\theta) = \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_\theta(\cdot | x)} \left[\frac{1}{|y|} \sum_{t=1}^{|y|} D(\pi_\theta(\cdot | x, y_{<t}) \| \pi_T(\cdot | x, y_{<t})) \right].$$

On-policy Self-Distillation. On-policy self-distillation (OPSD) removes the need for a separate teacher by embodying teacher and student policies from the same model under different contexts (Zhao et al., 2026; Hübottner et al., 2026; Shenfeld et al., 2026). Given a problem-solution pair $(x, y^*) \sim \mathcal{D}$, the teacher policy receives privileged information such as the reference answer or reasoning trace and is evaluated as $\pi_T = \pi_\theta(\cdot | x, y^*, y_{<t})$, with teacher and student sharing the parameters θ of the same model. The standard OPSD loss is

$$\mathcal{L}(\theta) = \mathbb{E}_{(x, y^*) \sim \mathcal{D}, y \sim \pi_\theta(\cdot | x)} \left[\frac{1}{|y|} \sum_{t=1}^{|y|} D(\pi_\theta(\cdot | x, y_{<t}) \| \text{sg}[\pi_\theta(\cdot | x, y^*, y_{<t})]) \right], \quad (1)$$

where $\text{sg}[\cdot]$ denotes the stop-gradient operator applied to the teacher branch. The choice of divergence is itself a design. Reverse KL is mode-seeking and is generally preferred for generative language-model distillation since it discourages the student from assigning probability to low-probability regions of the teacher, whereas forward KL is mode-covering (Gu et al., 2024; Lu & Thinking Machines Lab, 2025; Zhao et al., 2026; Jin et al., 2026; Jang et al., 2026). In practice, the full vocabulary $\mathcal{V}$ is widely adopted to reduce variance and stabilize gradient estimates relative to single-sample Monte Carlo estimation (Zhao et al., 2026; Yang et al., 2025). Having full access to both distributions makes the KL direction independent of the sampling distribution; see Jin et al. (2026) for detailed discussion.

4 Prefix Failure in Token-Level On-Policy Self-Distillation

4.1 Mixture of Distribution

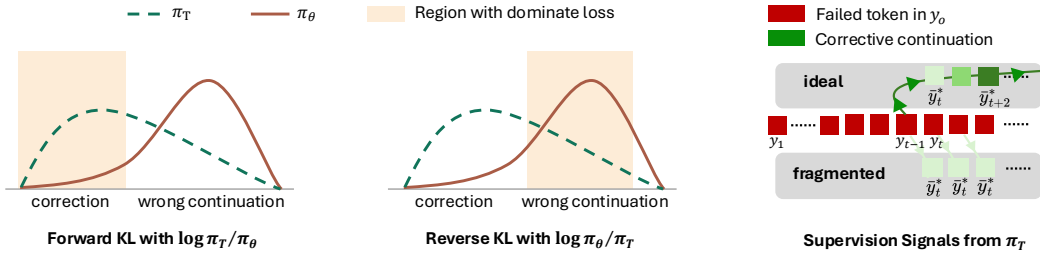


Figure 1: **Left and Middle:** Under prefix failure, the teacher distribution becomes a mixture with two modes. By their respective mode-covering and mode-seeking properties, forward KL is dominated by the correction-onset region, while reverse KL is dominated by the wrong-continuation region. **Right:** Supervision signal provided by the teacher under prefix failure.

Dense per-token KL relies on the teacher providing a reliable supervisory signal at every position along the student rollout. When the teacher faces a wrong reasoning path rolled out by the students, the supervision can turn noisy or even adversarial. Let $F(y_{o,<t})$ denote the *prefix-failure* event—the prefix $y_{o,<t}$ contains reasoning errors that contradict y^* and cannot be extended to y^* without

retraction or contradiction. Whenever $F(y_{o,<t})$ holds, the teacher becomes a mixture distribution with two modes, one that continues the failed prefix $y_{o,<t}$ for sequence consistency and another that pivots back toward y^* . This mixture structure turns the supposedly dense supervision into a noisy or even adversarial signal. A complementary failure mode arises on degenerate prefixes (e.g., repetition loops), where the teacher instead remains locally aligned with the student and the guidance signal vanishes entirely (Fu et al., 2026, Figure 3). Notably, prefix failure is unique to the on-policy dense signal training paradigm: SFT’s fixed trajectories stay aligned with y^* , and RLVR’s sparse end-of-trajectory rewards never touch intermediate prefixes.

The choice of KL direction interacts with prefix failure asymmetrically. Recovering from $F(y_{o,<t})$ requires a corrective continuation $\bar{y}_{\geq t}^* = (\bar{y}_t^*, \bar{y}_{t+1}^*, \dots, \bar{y}_{t+k-1}^*)^1$, generated autoregressively along the correction path, where $\bar{y}_t^*$ is a correction-onset token ($\bar{y}_t^* \in \{\text{Wait}, \text{Actually}, \dots\}$) and $\bar{y}_{t+1}^*, \dots, \bar{y}_{t+k-1}^*$ continue the recovery, with k denoting the length of this continuation. Under $F(y_{o,<t})$, the ideal teacher partially shifts mass from the natural continuation toward $\bar{y}_t^*$, while the student with high probability remains anchored on the wrong continuation. Forward KL $D(\pi_T \parallel \pi_\theta)$, weighted by π_T , is dominated by the correction-onset region; its mode-covering nature forces the student onto this out-of-distribution (OOD) mode. This can destabilize training and, in the worst case, lead to mode collapse (Zhao et al., 2026). In contrast, reverse KL $D(\pi_\theta \parallel \pi_T)$ is weighted by π_θ , so its mode-seeking behavior makes the loss dominated by the wrong-continuation region where the student already places high mass. Since $\pi_\theta(\bar{y}_t^*)$ is small, the supervision has little effect, causing updates to concentrate on the failing trajectory rather than the recovery tokens.

Prior work has empirically identified various failure modes of dense token-level KL training and proposed loss-design remedies. Many of these failures are consistent with the prefix-failure mechanism analyzed above, even when not labeled as such. Under forward KL, Zhao et al. (2026) clip each per-token loss by a threshold $c > 0$ to suppress the high-KL tokens that destabilize training. Under reverse KL, Fu et al. (2026) restrict the divergence to the teacher’s top- K vocabulary, and Xu et al. (2026b) reweight per-token losses by entropy and student-teacher disagreement, both aiming to amplify the informative token signal. In effect, those reverse-KL fixes attempt to induce mode-covering behavior within a mode-seeking objective, manually amplifying teacher modes (e.g., $\bar{y}_t^*$) that mode-seeking would otherwise underweight. These loss-level interventions, however, leave the underlying failed prefix in place. Fig. 1 (Left and Middle) depicts the mixture and KL asymmetry.

4.2 Can a Perfect Teacher Recover the Correction Path?

Even granting an ideal teacher, dense per-token KL is structurally limited because it is a per-position objective among student’s rollout. To make this precise, we trace the per-token OPSD loss back to its sequence-level origin, which makes the underlying mechanism more transparent. Define the per-token log-ratio $\delta_t := \log \pi_\theta(y_t \mid x, y_{<t}) - \log \pi_T$ with $\pi_T = \pi_\theta(y_t \mid x, y^*, y_{<t})$. Differentiating the sequence-level reverse KL of Eq. (1) (full derivation in Sec. A) yields the policy-gradient form

$$\nabla_\theta \mathcal{J}(\theta) = \mathbb{E}_{\substack{(x, y^*) \sim \mathcal{D} \\ y \sim \pi_\theta(\cdot \mid x)}} \left[\sum_{t=1}^{|y|} \left(\delta_t + \sum_{t'=t+1}^{|y|} \delta_{t'} \right) \nabla_\theta \log \pi_\theta(y_t \mid x, y_{<t}) \right], \quad (2)$$

where $-\delta_t$ acts as a token-level return. In practice, however, standard OPSD implementations (Lu & Thinking Machines Lab, 2025; Yang et al., 2025) do not optimize the sequence-level KL in Eq. (1); instead, they retain only the immediate log-ratio at each position, yielding the per-token surrogate

$$\nabla_\theta \mathcal{J}(\theta) = \mathbb{E}_{\substack{(x, y^*) \sim \mathcal{D} \\ y \sim \pi_\theta(\cdot \mid x)}} \left[\sum_{t=1}^{|y|} \delta_t \nabla_\theta \log \pi_\theta(y_t \mid x, y_{<t}) \right]. \quad (3)$$

We contrast Eq. (3) with the *perfect teacher* would induce by autoregressively unfolding $\bar{y}_{\geq t}^*$ along the correction path, delivering the supervision pairs

$$\{(y_{o,<t}, \bar{y}_t^*), ((y_{o,<t}, \bar{y}_t^*), \bar{y}_{t+1}^*), \dots, ((y_{o,<t}, \bar{y}_{t:t+k-1}^*), \bar{y}_{t+k-1}^*)\},$$

in which the context grows along the correction path itself. The corresponding ideal gradient is

$$g_{\text{ideal}} = - \sum_{i=1}^k \delta_i^{\text{ideal}} \cdot \nabla_\theta \log \pi_\theta(\bar{y}_{t+i-1}^* \mid x, y_{o,<t}, \bar{y}_{t:t+i-1}^*), \quad (4)$$

¹ $\bar{y}_{\geq t}^*$ is not the literal suffix of y^* , but any correct continuation conditioned on $y_{<t}$, i.e., $\text{Verify}(y_{<t} \cdot \bar{y}_{\geq t}^*, y^*) = 1$.

where $\delta_i^{\text{ideal}} := \log \pi_\theta(\bar{y}_{t+i-1}^* | x, y_{o,<t}, \bar{y}_{t:t+i-1}^*) - \log \pi_T(\bar{y}_{t+i-1}^* | x, y_{o,<t}, \bar{y}_{t:t+i-1}^*)$. Yet under dense KL the contexts are dictated by the *frozen* student trajectory, not by the unfolding correction. At position t , the teacher conveys $\bar{y}_t^*$ given $y_{o,<t}$, and the student updates its parameters to favor $\bar{y}_t^*$. At position $t+1$, however, the teacher’s supervision is conditioned on $y_{o,<t+1} = (y_{o,<t}, y_{o,t})$ (the original failed trajectory’s own continuation), not on the correction path $(y_{o,<t}, \bar{y}_t^*)$. Because every subsequent prefix the teacher sees is still anchored in the original failure rather than the unfolding correction, the teacher is left repeatedly recommending the same correction-onset token. The supervision pairs delivered to the student therefore form the fragmented sequence

$$\{(y_{o,<t}, \bar{y}_t^*), (y_{o,<t+1}, \bar{y}_t^*), \dots, (y_{o,<t+k-1}, \bar{y}_t^*)\},$$

in which the context grows along the wrong continuation while the target remains stuck at $\bar{y}_t^*$. The corresponding gradient

$$g_{\text{frag}} = - \sum_{i=0}^{k-1} \delta_i^{\text{frag}} \cdot \nabla_\theta \log \pi_\theta(\bar{y}_t^* | x, y_{o,<t+i}),$$

with $\delta_i^{\text{frag}} := \log \pi_\theta(\bar{y}_t^* | x, y_{o,<t+i}) - \log \pi_T(\bar{y}_t^* | x, y_{o,<t+i})$, evaluates the score function at a completely different set of (context, token) pairs than g_{ideal} . The two pair sets share only their first element $(y_{o,<t}, \bar{y}_t^*)$; beyond it, g_{ideal} propagates supervision along $(y_{o,<t}, \bar{y}_t^*, \bar{y}_{t+1}^*, \dots)$ while g_{frag} accumulates supervision along $(y_{o,<t}, y_{o,t}, y_{o,t+1}, \dots)$. The two trajectories diverge after a single step and never re-intersect:

$$\underbrace{\{((y_{o,<t}, \bar{y}_{t:t+i-1}^*), \bar{y}_{t+i-1}^*)\}_{i=1}^k}_{\text{required by recovery}} \cap \underbrace{\{(y_{o,<t+i}, \bar{y}_t^*)\}_{i=0}^{k-1}}_{\text{delivered}} = \{(y_{o,<t}, \bar{y}_t^*)\}.$$

The privileged information y^* thus stays trapped in per-position marginals. Dense KL keeps recommending $\bar{y}_t^*$ on ever-deepening wrong-continuation contexts but cannot supervise the multi-step unfolding of $\bar{y}_{\geq t}^*$, and loss-level interventions only reweight terms within the visited pair set $\{(y_{o,<t+i}, \bar{y}_t^*)\}$ rather than move the gradient onto the correction-path pair set. The g_{frag} signal is not useless, however; biasing the student toward $\bar{y}_t^*$ can trigger self-correction, though unfolding the full $\bar{y}_{\geq t}^*$ is bounded by the student’s capacity. Our method SRD (Sec. 5) recovers g_{ideal} by supervising the per-token KL along a refined trajectory generated under privileged context, so the supervision contexts grow along the correction path itself rather than the frozen failed prefix.

4.3 Experimental Validation of Prefix Failure

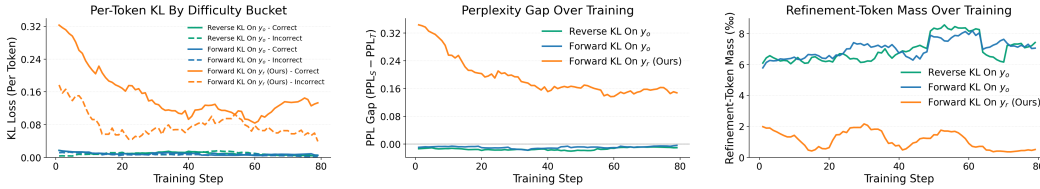


Figure 2: Empirical observations of prefix failure under standard OPSD (y_o , forward and reverse KL). **Left:** Per-token KL by correct/incorrect rollouts. **Middle:** Teacher-student perplexity gap. **Right:** Teacher’s epistemic mass.

We empirically verify the prefix failure mechanism through three measurements over OPSD training on student rollouts y_o under both forward and reverse KL (Fig. 2). A third curve (*ours*) is included for reference, corresponding to the method introduced in Sec. 5.

Distribution-level supervision collapse (left). We split the per-token KL between π_T and π_S by stage-1 verifier outcome (correct vs. incorrect) into token-weighted means D^{correct} and $D^{\text{incorrect}}$. Both stay pinned near zero on y_o , so π_T and π_S remain aligned and dense KL delivers no signal once $F(y_{o,<t})$ saturates. Notably, even $D^{\text{incorrect}}$ stays near zero, indicating that the teacher with high probability collapses onto the student’s failure rather than diverging to correct it.

Perplexity gap collapse (middle). We measure teacher and student perplexities token-wise on the same response mask under teacher-forced decoding, $\text{PPL}_S = \exp(-1/|y_o| \sum_t \log \pi_S(y_{o,t} | y_{o,<t}))$

and PPL_T defined analogously with π_T . The gap $\text{PPL}_S - \text{PPL}_T$ stays near zero throughout training, so the privileged condition y^* delivers essentially vanishing incremental supervision over what the student already represents on its own rollouts.

Epistemic-token mass (right). The teacher places 6–8% of its per-position mass on 16 epistemic onset tokens throughout training, matching the $\bar{y}_t^*$ -repeat signature of g_{frag} predicted in Sec. 4.2. Strikingly, student and teacher top-16 tokens already absorb 97 – 99 % of total probability mass (Li et al., 2026b, Figure 18), so this allocation commands a disproportionately dominant share of the remaining 1 – 3 % residual budget. The same metric on y_r (*ours*) collapses below 2%, confirming concentration is tied to failed-prefix conditioning rather than a universal teacher property.

5 Self-Refined Distillation

The previous section identifies *prefix failure* as one of the central bottlenecks in OPSD. With prefix failure, dense supervision becomes noisy and can even fail to guide the student. Most existing mitigations operate through loss design and leave the offending prefix unchanged. Directly optimizing prefix failure is intractable since identifying the failure index requires enumerating $\mathcal{O}(|\mathcal{V}|^k)$ with following reasoning paths of length k . We therefore relax the target to a trajectory-level surrogate that maximizes the expected verifier-pass rate over the dataset $\mathcal{D}$ under the support constraint of π_θ :

$$\max_q \mathbb{E}_{(x, y^*) \sim \mathcal{D}} [\Pr_{y \sim q(\cdot | x, \cdot)} \{\text{Verify}(y, y^*) = 1\}] \quad \text{s.t.} \quad \pi_\theta(y | x) > 0 \quad (5)$$

We note that this is a *trajectory-level* distribution support constraint and the optimization objective form mirrors a standard RLVR objective. We emphasize, however, that θ is *not* the direct optimization variable: the equation instead specifies the goal of an upstream process before the OPSD gradient update with θ , namely to construct trajectories that attain higher verifier-pass rates while remaining within the current student’s support. These trajectories then serve as the supervision for the subsequent update of the OPSD through the loss of the standard distribution matching in Eq. (1).

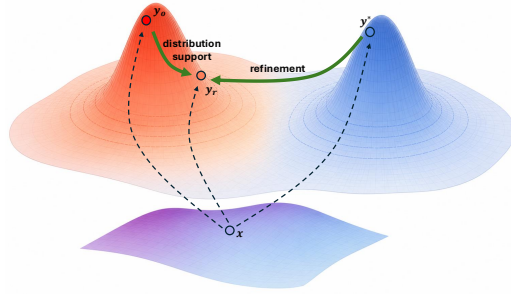
Two extreme choices of q illustrate the tension between objective and constraint. (I) Setting $q(\cdot | x, \cdot) = \pi_\theta(\cdot | x)$ fails back to standard OPSD. The on-policy constraint is satisfied by construction, but this choice fails to mitigate *prefix failure* beyond current OPSD algorithms, since the supervision trajectories are still drawn from π_θ . Repeated sampling refines this choice by drawing rollouts and retaining only the verifier-passing ones (Brown et al., 2024; Stein et al., 2026), straining inference budgets linearly and yielding nothing on questions the student cannot already solve. Any improvement in pass rates depends entirely on the stochasticity of base LLM decoding, with no direct optimization of Eq. (5). (II) Setting $q(\cdot | x, \cdot) = \pi^*$, i.e., the expert policy that produces y^* , attains $\text{Verify} = 1$ exactly but generally violates the on-policy support constraint, so this choice falls outside the feasible set and breaks the on-policy character that OPSD depends on.

To move beyond the two extremes above, we propose *Self-Refined Distillation* (SRD), which operates at the trajectory level to optimize Eq. (5) by first drawing a raw on-policy rollout $y_o \sim \pi_\theta(\cdot | x)$, then constructing a refined trajectory y_r via:

$$y_r \sim q(\cdot | x, \cdot) := \pi_\theta(\cdot | x, y_o, y^*).$$

Crucially, conditioning on y_o anchors y_r to the reasoning patterns π_θ has already demonstrated, i.e., within the policy support, while π_θ guided by y^* rewrites the erroneous portions to directly mitigate prefix failure. To our knowledge, this is the first design that explicitly optimizes Eq. (5) while respecting the on-policy constraint without prohibitive additional compute overhead. A refined trajectory $y_r \sim q(\cdot | x, y_o, y^*)$ is then used as supervision for the subsequent OPSD update. Per-token KL along y_r reduces exposure to the bimodal teacher mixture (Sec. 4.1) and recovers the ideal gradient g_{ideal} (Sec. 4.2), since supervision contexts grow along the refined trajectory rather than the raw rollout y_o at higher risk of prefix failure.

SRD also boosts the student’s exploration beyond standard OPSD. On a correct y_o , standard OPSD provides little new signal: it tends to merely reinforces the high-probability solution path the student already produces due to the fragmented gradient. In contrast, y_r is drawn under the privileged condition y^* and surfaces alternative valid derivations of the same answer, i.e., paths consistent with y^* but rarely sampled from $\pi_\theta(\cdot | x)$ alone, thereby expanding the set of correct reasoning trajectories the student is supervised on. SRD therefore adds value in both regimes: mitigating failed prefixes when y_o is wrong, and broadening the supervision distribution when y_o already succeeds.



Algorithm 1: Self-Refined Distillation

Require: minibatch $\mathcal{B} \subset \mathcal{D}$, policy π_θ

- 1: **for** $(x, y^*) \in \mathcal{B}$ **do**
- 2: $y_o \sim \pi_\theta(\cdot | x)$
- 3: $y_r \sim \pi_\theta(\cdot | x, y_o, y^*)$
- 4: Update θ by Eq. (6) on y_r
- 5: **end for**

Figure 3: **Left:** SRD refines y_o to y_r under privileged context y^* . **Right:** SRD pseudocode.

249 The training-data analysis in Sec. 6.4 confirms this (e.g., the correct subset of y_r exhibits a low-length
 250 mode absent in y_o) and translates into Pass@ k gains in Sec. 6.3.

251 Concretely, given y_r , we instantiate the distillation loss as the forward KL with full vocabulary
 252 matching over $\mathcal{V}$, which provides mode-covering supervision and stabilizes gradient estimates:

$$\mathcal{L}(\theta) = \mathbb{E}_{\substack{(x, y^*) \sim \mathcal{D} \\ y_o \sim \pi_\theta(\cdot | x) \\ y_r \sim \pi_\theta(\cdot | x, y_o, y^*)}} \left[\frac{1}{|y_r|} \sum_{t=1}^{|y_r|} D(\text{sg}[\pi_\theta(\cdot | x, y_o, y^*, y_{r, < t})] \parallel \pi_\theta(\cdot | x, y_{r, < t})) \right], \quad (6)$$

253 The exact prompt template is given in Sec. C.4; Fig. 3 and Algorithm 1 illustrate the full procedure.
 254 Fig. 2 confirms that these design choices alleviate the failures observed on y_o . Beyond the per-token
 255 KL recovery on y_r (Left, discussed above), the teacher-student perplexity gap opens on y_r (Middle),
 256 restoring the incremental supervision, and the teacher’s epistemic onset mass decreases 3x less than
 257 y_o . KL and the PPL gap both decrease steadily as epistemic onset concentration fades, indicating that
 258 the privileged signal is genuinely transmitted to the student. The flat y_o curves, in contrast, reflect
 259 collapsed learning signals under prefix failure.

260 6 Experiments

261 We evaluate SRD against four dense-KL baselines on five competition-math benchmarks. We answer
 262 three questions. (I) Does SRD improve exploitation (Avg@16) over the base model (Sec. 6.2)?
 263 (II) Does SRD improve exploration (Pass@16) beyond the base model’s support (Sec. 6.3)? (III)
 264 How does y_r differ from y_o on the training corpus (Sec. 6.4)? Sec. B.1 provides a detailed training-
 265 trajectory analysis comparing y_o and y_r across model scales and reference-quality regimes; Sec. B.2
 266 stratifies AMOBench test rollouts by base-model difficulty bucket to localize where SRD’s Pass@16
 267 gain originates; Sec. B.3 ablates the training corpus by initial-rollout correctness on Qwen3-8B. Full
 268 experimental details are given in Sec. C.

269 6.1 Experiments Setup and Baselines

270 **Models and training data.** We use Qwen3-4B-Instruct-2507 and Qwen3-8B (both with thinking
 271 enabled) as base models (Yang et al., 2025), and train on the DeepScaleR corpus of roughly 40K
 272 math problems with verified answers (Luo et al., 2025). Following the OPSD setup (Sec. 3), the
 273 same backbone embodies teacher and student via the privileged context y^* , with no separate teacher
 274 network. For every prompt x , we first draw a raw rollout $y_o \sim \pi_\theta(\cdot | x)$ from the current student;
 275 for SRD we additionally draw the refined trajectory y_r via the rewrite prompt in Sec. C.4, while the
 276 dense-KL baselines train directly on y_o . We use LoRA adapters (rank 64, $\alpha=128$) on all attention
 277 and MLP linear layers and train for one epoch with bfloat16 and gradient checkpointing. Full
 278 hyperparameters and the data-generation pipeline are given in Sec. C.

279 **Baselines.** All baselines minimize per-token KL between the student and the privileged-context
 280 teacher over the full Qwen3 vocabulary ($|\mathcal{V}| \approx 152\text{K}$), differing only in KL direction and the
 281 per-token correction applied to mitigate prefix-failure symptoms. *Forward KL* ($D(\pi_T \parallel \pi_\theta)$) is

Table 1: Avg@16 results (%). Per-question average over 16 sampled completions. Parentheses report absolute percentage-point changes from the corresponding base model. Bold marks best performance within each model block, and red cells mark drops below the corresponding base model.

Method	Traj.	AIME24	AIME25	HMMT25	BeyondAIME	AMOBench
<i>Qwen3-4B-Instruct-2507 (w/ thinking)</i>						
Base	—	63.1	48.2	31.3	32.2	10.6
+Forward KL	y_o	60.3 (-2.8)	48.8 (+0.6)	30.8 (-0.5)	32.3 (+0.1)	9.5 (-1.1)
+Forward KL w/ Clip	y_o	62.1 (-1.0)	49.2 (+1.0)	27.9 (-3.4)	32.4 (+0.2)	9.3 (-1.3)
+Reverse KL	y_o	58.4 (-4.7)	48.8 (+0.6)	30.0 (-1.3)	31.6 (-0.6)	10.1 (-0.5)
+Reverse KL w/ Top-K	y_o	63.0 (-0.1)	49.0 (+0.8)	31.0 (-0.3)	32.3 (+0.1)	9.6 (-1.0)
+SRD (ours)	y_r	63.1 (+0.0)	49.4 (+1.2)	32.1 (+0.8)	32.7 (+0.5)	11.2 (+0.6)
<i>Qwen3-8B (w/ thinking)</i>						
Base	—	76.5	66.7	41.5	41.6	15.9
+Forward KL	y_o	74.8 (-1.7)	65.4 (-1.3)	40.3 (-1.2)	39.6 (-2.0)	15.7 (-0.2)
+Forward KL w/ Clip	y_o	76.5 (+0.0)	68.3 (+1.6)	42.5 (+1.0)	40.2 (-1.4)	15.9 (+0.0)
+Reverse KL	y_o	75.4 (-1.1)	68.2 (+1.5)	44.3 (+2.8)	40.8 (-0.8)	16.8 (+0.9)
+Reverse KL w/ Top-K	y_o	75.6 (-0.9)	68.5 (+1.8)	44.4 (+2.9)	41.9 (+0.3)	15.2 (-0.7)
+SRD (ours)	y_r	76.5 (+0.0)	69.2 (+2.5)	44.5 (+3.0)	42.8 (+1.2)	17.3 (+1.4)

the unmodified mode-covering baseline. *Forward KL w/ Clip* caps each token’s KL contribution at $c=0.06$, reproducing the OPSD recipe of Zhao et al. (2026). *Reverse KL* ($D(\pi_\theta \parallel \pi_T)$) is the unmodified mode-seeking baseline. *Reverse KL w/ Top-K* restricts the loss to the teacher’s top-32 support and renormalizes both distributions inside it, reproducing the local-support-matching objective of Fu et al. (2026). All four baselines train on the raw on-policy rollout $y_o \sim \pi_\theta(\cdot \mid x)$; SRD instead trains on the refined trajectory y_r defined in Sec. 5.

Evaluation. We report Avg@16 and Pass@16 on AIME24 (Mathematical Association of America, 2024), AIME25 (Mathematical Association of America, 2025), HMMT25 (HMMT, 2025), BeyondAIME (Seed, 2025), and AMOBench (An et al., 2025), following the Qwen3 evaluation setup (Yang et al., 2025). For each test question we draw $K=16$ completions and grade them with an external verifier; Avg@16 averages the K binary outcomes per question, and Pass@16 is the per-question indicator that at least one of the K samples is correct, both then averaged over the test set. Formal definitions are in Sec. C.2.

6.2 SRD Sharpens Single-Attempt Accuracy

Tab. 1 reports Avg@16. SRD is best on every benchmark at both scales and never drops below base (no red cells in SRD rows). AIME24 and AIME25 are largely saturated at this scale (SRD matches base on AIME24, all methods within ~ 2 pp on AIME25), so the contrast with loss-design baselines is sharpest on the other three benchmarks. Three of four baselines regress on at least one benchmark (e.g., Reverse KL -5.0 pp on AIME24-4B, Forward KL w/ Clip -3.4 pp on HMMT25-4B, Forward KL -2.0 pp on BeyondAIME-8B), reflecting the prefix-failure asymmetry of Sec. 4, while SRD delivers consistent gains (e.g., $+3.0$ pp on HMMT25 and $+1.4$ pp on AMOBench at 8B). Notably, the regressions span both KL directions and both per-token corrections (clipping and top- K truncation), indicating that prefix-failure-induced drops cannot be repaired by tuning the loss alone. On Qwen3-4B in particular, every dense-KL baseline regresses on at least one of HMMT25/BeyondAIME/AMOBench, whereas SRD delivers $+0.5$ to $+1.2$ pp gains on every non-saturated column. Fixing the offending trajectory is more reliable than reweighting the loss it induces.

6.3 SRD Expands Reasoning Coverage

Tab. 2 reports Pass@16, where SRD’s trajectory-level rewrite separates most clearly from per-token interventions. On Qwen3-8B, SRD lifts AMOBench from 41.0% to 61.5% ($+20.5$ pp, a 50% relative gain) and HMMT25 from 66.7% to 73.3% ($+6.6$ pp), while the strongest dense-KL baseline on AMOBench stops at 51.3% and three of four baselines drop on AIME24. On Qwen3-4B-Instruct-2507, SRD posts $+5.0$ pp on BeyondAIME and $+10.7$ pp on AMOBench, again top of every column. On AMOBench, SRD solves 9/23 (39%) of the questions the base fails on all 16 attempts, nearly doubling the best baseline (+Forward KL at 22%); these are unreachable for the base at $K=16$, so the gain is genuine support expansion. See more detailed test rollout analysis in Sec. B.2.

Table 2: Pass@16 results (%). Parentheses report absolute percentage-point changes from the corresponding base model. Bold marks best performance within each model block, and red cells mark drops below the corresponding base model.

Method	Traj.	AIME24	AIME25	HMMT25	BeyondAIME	AMOBench
<i>Qwen3-4B-Instruct-2507 (w/ thinking)</i>						
Base	–	83.3	76.7	50.0	59.0	23.1
+Forward KL	y_o	86.7 (+3.4)	76.7 (+0.0)	53.3 (+3.3)	58.0 (-1.0)	33.3 (+10.2)
+Forward KL w/ Clip	y_o	86.7 (+3.4)	76.7 (+0.0)	46.7 (-3.3)	62.0 (+3.0)	28.2 (+5.1)
+Reverse KL	y_o	86.7 (+3.4)	76.7 (+0.0)	50.0 (+0.0)	60.0 (+1.0)	28.2 (+5.1)
+Reverse KL w/ Top-K	y_o	86.7 (+3.4)	76.7 (+0.0)	56.7 (+6.7)	57.0 (-2.0)	25.6 (+2.5)
+SRD (ours)	y_r	86.7 (+3.4)	80.0 (+3.3)	56.7 (+6.7)	64.0 (+5.0)	33.8 (+10.7)
<i>Qwen3-8B (w/ thinking)</i>						
Base	–	90.0	83.3	66.7	66.0	41.0
+Forward KL	y_o	86.7 (-3.3)	82.7 (-0.6)	73.3 (+6.6)	68.0 (+2.0)	51.3 (+10.3)
+Forward KL w/ Clip	y_o	90.0 (+0.0)	86.7 (+3.4)	70.0 (+3.3)	61.0 (-5.0)	43.6 (+2.6)
+Reverse KL	y_o	86.7 (-3.3)	86.7 (+3.4)	73.3 (+6.6)	64.0 (-2.0)	51.3 (+10.3)
+Reverse KL w/ Top-K	y_o	86.7 (-3.3)	86.6 (+3.3)	66.6 (-0.1)	69.0 (+3.0)	38.5 (-2.5)
+SRD (ours)	y_r	90.0 (+0.0)	86.7 (+3.4)	76.3 (+6.6)	68.0 (+2.0)	61.5 (+20.5)

6.4 Training Trajectory Analysis

Fig. 4 contrasts y^* , the raw rollout y_o , and the refined trajectory y_r on Qwen3-8B over the training corpus, optimizing Eq. (5) indirectly within support. The verifier-pass rate improves from 65.8% on y_o to 81.4% on y_r (+15.6pp), and the length distribution compresses by roughly $9\times$ (median 7.7K $\rightarrow$ 0.88K) toward the reference scale (y^* median $\sim$ 0.49K), with y_r remaining within the student’s support since it is sampled from π_θ itself. We highlight that the same $\sim 9\times$ compression also applies to the *correct* half of y_o , surfacing a low-length mode in y_r that y_o does not produce. This means even on questions the student already solves, SRD exposes it to alternative, shorter derivations under y^* guidance, an additional source of supervision diversity; see Sec. B.1 for the full analysis. The same compression also cuts training wall-clock by roughly 60% on Qwen3-8B, which offsets the extra y_r sampling cost (Sec. C.3). By supervising on y_r rather than the raw rollout y_o , SRD also softens the length-inflation regime that Luo et al. (2026) document for OPD training on long on-policy rollouts.

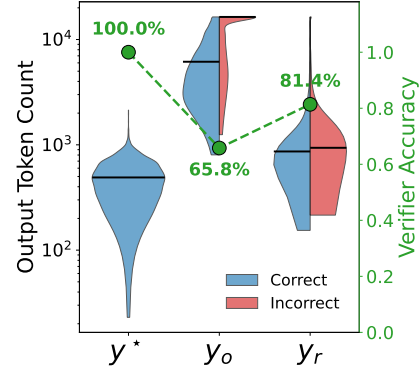


Figure 4: Length distribution and verifier accuracy on Qwen3-8B.

7 Conclusion

We identify *prefix failure* as a structural limitation of on-policy (self)-distillation paradigms, where the per-token KL evaluated along the student’s frozen rollout induces a bimodal teacher mixture and a fragmented gradient that loss-level fixes leave structurally intact. To address it, we propose SRD, a trajectory-level refinement that draws a refined trajectory y_r under privileged context and supervises the per-token KL along y_r , recovering the ideal supervision-pair structure while remaining on-policy. Across five competition-math benchmarks for Qwen3-4B and Qwen3-8B, SRD attains the best Avg@16 on every benchmark and substantial Pass@16 gains, including solving 9/23 of base-unreachable AMOBench questions and nearly doubling the strongest baseline.

Limitations. First, SRD adds one extra sampling pass per problem to construct y_r . This overhead is largely offset by the roughly $9\times$ length compression of y_r relative to y_o , which proportionally reduces the per-step training compute; on Qwen3-8B the end-to-end wall-clock matches the dense-KL baselines (Sec. C.3). Second, SRD relies on the base model having sufficient capacity to produce a correct continuation under privileged context and thus improve Eq. (5). When $\pi_\theta(\cdot | x, y_o, y^*)$ cannot reach a verifier-passing trajectory, y_r collapses toward y_o and the refinement signal vanishes, leaving SRD with no advantage over baselines in that regime.

References

- Agarwal, R., Vieillard, N., Zhou, Y., Stanczyk, P., Garea, S. R., Geist, M., and Bachem, O. On-policy distillation of language models: Learning from self-generated mistakes. In *The twelfth international conference on learning representations*, 2024.
- An, S., Cai, X., Cao, X., Li, X., Lin, Y., Liu, J., Lv, X., Ma, D., Wang, X., Wang, Z., et al. Amobench: Large language models still struggle in high school math competitions. *arXiv preprint arXiv:2510.26768*, 2025.
- Brown, B., Juravsky, J., Ehrlich, R., Clark, R., Le, Q. V., Ré, C., and Mirhoseini, A. Large language monkeys: Scaling inference compute with repeated sampling. *arXiv preprint arXiv:2407.21787*, 2024.
- DeepSeek-AI. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.
- DeepSeek-AI. Deepseek-v4: Towards highly efficient million-token context intelligence. Technical report, DeepSeek-AI, 2026. URL <https://huggingface.co/deepseek-ai/DeepSeek-V4-Pro>. Technical report and model card.
- Fu, Y., Huang, H., Jiang, K., Zhu, Y., and Zhao, D. Revisiting on-policy distillation: Empirical failure modes and simple fixes. *arXiv preprint arXiv:2603.25562*, 2026.
- Gu, Y., Dong, L., Wei, F., and Huang, M. MiniLLM: Knowledge distillation of large language models. In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024. URL <https://openreview.net/forum?id=5h0qf71BZZ>.
- Hinton, G., Vinyals, O., and Dean, J. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015.
- HMMT. Harvard-MIT mathematics tournament, february 2025. <https://www.hmmt.org/www/archive/results>, 2025.
- Hübötter, J., Lübeck, F., Behric, L., Baumann, A., Bagatella, M., Marta, D., Hakimi, I., Shenfeld, I., Buening, T. K., Guestrin, C., et al. Reinforcement learning via self-distillation. *arXiv preprint arXiv:2601.20802*, 2026.
- Jang, I., Yeom, J., Yeo, J., Lim, H., and Kim, T. Stable on-policy distillation through adaptive target reformulation. *arXiv preprint arXiv:2601.07155*, 2026.
- Jin, W., Min, T., Yang, Y., Kadhe, S. R., Zhou, Y., Wei, D., Baracaldo, N., and Lee, K. Entropy-aware on-policy distillation of language models. *arXiv preprint arXiv:2603.07079*, 2026.
- Kim, J., Luo, X., Kim, M., Lee, S., Kim, D., Jeon, J., Li, D., and Yang, Y. Why does self-distillation (sometimes) degrade the reasoning capability of llms? *arXiv preprint arXiv:2603.24472*, 2026.
- Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*, 2023.
- Li, G., Yang, T., Fang, J., Song, M., Zheng, M., Guo, H., Zhang, D., Wang, J., and Chua, T.-S. Unifying group-relative and self-distillation policy optimization via sample routing, 2026a. URL <https://arxiv.org/abs/2604.02288>.
- Li, Y., Zuo, Y., He, B., Zhang, J., Xiao, C., Qian, C., Yu, T., Gao, H.-a., Yang, W., Liu, Z., et al. Rethinking on-policy distillation of large language models: Phenomenology, mechanism, and recipe. *arXiv preprint arXiv:2604.13016*, 2026b.
- Lu, K. and Thinking Machines Lab. On-policy distillation. Thinking Machines Lab: Connectionism, 2025. <https://thinkingmachines.ai/blog/on-policy-distillation>.
- Luo, F., Chuang, Y.-N., Wang, G., Xu, Z., Han, X., Zhang, T., and Braverman, V. Demystifying opd: Length inflation and stabilization strategies for large language models, 2026. URL <https://arxiv.org/abs/2604.08527>.

401 Luo, M., Tan, S., Wong, J., Shi, X., Tang, W. Y., Roongta, M., Cai, C., Luo, J., Zhang, T., Li, L. E.,
402 et al. Deepscaler: Surpassing o1-preview with a 1.5 b model by scaling rl. *Notion Blog*, 3(5), 2025.

403 Mathematical Association of America. AIME 2024 problems. https://artofproblemsolving.com/wiki/index.php/AIME_Problems_and_Solutions, 2024.

405 Mathematical Association of America. AIME 2025 problems. https://artofproblemsolving.com/wiki/index.php/AIME_Problems_and_Solutions, 2025.

407 Penalzoa, E., Vattikonda, D., Gontier, N., Lacoste, A., Charlin, L., and Caccia, M. Privileged
408 information distillation for language models. *arXiv preprint arXiv:2602.04942*, 2026.

409 Seed, B. Seed1.5-Thinking: Advancing superb reasoning models with reinforcement learning.
410 <https://huggingface.co/datasets/ByteDance-Seed/BeyondAIME>, 2025.

411 Shenfeld, I., Damani, M., Hübotter, J., and Agrawal, P. Self-distillation enables continual learning.
412 *arXiv preprint arXiv:2601.19897*, 2026.

413 Sheng, G., Zhang, C., Ye, Z., Wu, X., Zhang, W., Zhang, R., Peng, Y., Lin, H., and Wu, C. HybridFlow:
414 A flexible and efficient RLHF framework. *arXiv preprint arXiv:2409.19256*, 2024.

415 Shi, T., Chen, S., Jiang, B., Song, L., Yang, L., and Zhao, J. Experiential reinforcement learning.
416 *arXiv preprint arXiv:2602.13949*, 2026.

417 Song, M. and Zheng, M. A survey of on-policy distillation for large language models. *arXiv preprint*
418 *arXiv:2604.00626*, 2026.

419 Stein, A., Huang, F., and Goldstein, T. Gates: Self-distillation under privileged context with consensus
420 gating, 2026. URL <https://arxiv.org/abs/2602.20574>.

421 Wang, H., Wang, G., Xiao, H., Zhou, Y., Pan, Y., Wang, J., Xu, K., Wen, Y., Ruan, X., Chen,
422 X., et al. Skill-sd: Skill-conditioned self-distillation for multi-turn llm agents. *arXiv preprint*
423 *arXiv:2604.10674*, 2026.

424 Xiao, B., Xia, B., Yang, B., Gao, B., Shen, B., Zhang, C., He, C., Lou, C., Luo, F., Wang, G., et al.
425 Mimo-v2-flash technical report. *arXiv preprint arXiv:2601.02780*, 2026.

426 Xu, Y., Sang, H., Zhou, Z., He, R., and Wang, Z. Paced: Distillation and on-policy self-distillation at
427 the frontier of student competence, 2026a. URL <https://arxiv.org/abs/2603.11178>.

428 Xu, Y., Sang, H., Zhou, Z., He, R., Wang, Z., and Geramifard, A. Tip: Token importance in on-policy
429 distillation. *arXiv preprint arXiv:2604.14084*, 2026b.

430 Yang, A., Li, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Gao, C., Huang, C., Lv, C., et al.
431 Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

432 Yang, C., Qin, C., Si, Q., Chen, M., Gu, N., Yao, D., Lin, Z., Wang, W., Wang, J., and Duan, N.
433 Self-distilled rlvr, 2026a. URL <https://arxiv.org/abs/2604.03128>.

434 Yang, W., Liu, W., Xie, R., Yang, K., Yang, S., and Lin, Y. Learning beyond teacher: Generalized
435 on-policy distillation with reward extrapolation. *arXiv preprint arXiv:2602.12125*, 2026b.

436 Ye, T., Dong, L., Wu, X., Huang, S., and Wei, F. On-policy context distillation for language models.
437 *arXiv preprint arXiv:2602.12275*, 2026.

438 Zeng, A., Lv, X., Hou, Z., Du, Z., Zheng, Q., Chen, B., Yin, D., Ge, C., Huang, C., Xie, C., et al.
439 Glm-5: from vibe coding to agentic engineering. *arXiv preprint arXiv:2602.15763*, 2026.

440 Zhao, S., Xie, Z., Liu, M., Huang, J., Pang, G., Chen, F., and Grover, A. Self-distilled reasoner:
441 On-policy self-distillation for large language models. *arXiv preprint arXiv:2601.18734*, 2026.

Appendix: Self-Refined Distillation

A Derivation of the OPD Policy Gradient

We derive the policy-gradient form [Eq. \(3\)](#) of the on-policy distillation gradient, expressing it in the dense form used in our analysis. The derivation parallels [Yang et al. \(2026b\)](#) and is reproduced here for completeness using the notation of [Sec. 3](#). Throughout, π_T denotes the teacher; under OPSD, $\delta_t := \log \pi_\theta(y_t | x, y_{<t}) - \log \pi_T$ with $\pi_T = \pi_\theta(y_t | x, y^*, y_{<t})$.

Step 1 (KL as a log-ratio expectation). Starting from the sequence-level reverse KL $\mathcal{J}(\theta) = \mathbb{E}[D(\pi_\theta(y | x) || \pi_T(y | x))]$, the objective expands to

$$\mathcal{J}(\theta) = \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_\theta(\cdot | x)} [\log \pi_\theta(y | x) - \log \pi_T(y | x)],$$

where the dependence on θ enters through both the sampling distribution and the integrand.

Step 2 (product rule and score-function trick). Differentiating under the expectation gives

$$\begin{aligned} \nabla_\theta \mathcal{J}(\theta) &= \mathbb{E}_x \left[\sum_y (\nabla_\theta \pi_\theta(y | x)) (\log \pi_\theta(y | x) - \log \pi_T(y | x)) \right. \\ &\quad \left. + \sum_y \pi_\theta(y | x) \nabla_\theta \log \pi_\theta(y | x) \right]. \end{aligned}$$

The second sum vanishes because

$$\sum_y \pi_\theta(y | x) \nabla_\theta \log \pi_\theta(y | x) = \sum_y \nabla_\theta \pi_\theta(y | x) = \nabla_\theta 1 = 0.$$

Using $\nabla_\theta \pi_\theta = \pi_\theta \nabla_\theta \log \pi_\theta$, the remaining term becomes

$$\nabla_\theta \mathcal{J}(\theta) = \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_\theta(\cdot | x)} [(\log \pi_\theta(y | x) - \log \pi_T(y | x)) \nabla_\theta \log \pi_\theta(y | x)]. \quad (7)$$

Step 3 (autoregressive decomposition). Factor $\log \pi_\theta(y | x) = \sum_t \log \pi_\theta(y_t | x, y_{<t})$ and similarly for π_T . [Eq. \(7\)](#) expands to

$$\mathbb{E}_{x,y} \left[\sum_{t=1}^{|y|} \sum_{t'=1}^{|y|} \underbrace{(\log \pi_\theta(y_{t'} | x, y_{<t'}) - \log \pi_T(y_{t'} | x, y_{<t'}))}_{\delta_{t'}} \nabla_\theta \log \pi_\theta(y_t | x, y_{<t}) \right].$$

Step 4 (causality, future tokens do not contribute). For any $t' < t$, $\delta_{t'}$ is measurable with respect to $(x, y_{<t})$, and conditioning on this prefix yields

$$\mathbb{E}_{y_t \sim \pi_\theta(\cdot | x, y_{<t})} [\nabla_\theta \log \pi_\theta(y_t | x, y_{<t})] = \sum_{y_t} \nabla_\theta \pi_\theta(y_t | x, y_{<t}) = \nabla_\theta 1 = 0,$$

so all cross terms with $t' < t$ vanish in expectation.

Step 5 (final form). Retaining only $t' \geq t$ recovers [Eq. \(3\)](#),

$$\nabla_\theta \mathcal{J}(\theta) = \mathbb{E}_{x,y} \left[\sum_{t=1}^{|y|} \left(\sum_{t'=t}^{|y|} \delta_{t'} \right) \nabla_\theta \log \pi_\theta(y_t | x, y_{<t}) \right].$$

The bracketed quantity $-\sum_{t' \geq t} \delta_{t'}$ acts as a return-to-go for token y_t . Following common practice ([Agarwal et al., 2024](#); [Lu & Thinking Machines Lab, 2025](#)), applying a discount factor of 0 retains only the term at $t' = t$, which gives the per-token surrogate

$$\nabla_\theta \mathcal{J}(\theta) \approx \mathbb{E}_{x,y} \left[\sum_{t=1}^{|y|} (\log \pi_\theta(y_t | x, y_{<t}) - \log \pi_T(y_t | x, y_{<t})) \nabla_\theta \log \pi_\theta(y_t | x, y_{<t}) \right],$$

which is the gradient of the per-token KL loss in [Sec. 3](#), and, with the privileged-context substitution $\pi_T(\cdot | x, y_{<t}) = \pi_\theta(\cdot | x, y^*, y_{<t})$, of the OPSD loss in [Eq. \(1\)](#).

B Additional Experiments

This section provides three complementary analyses beyond the dataset-averaged numbers in Tabs. 1 and 2. Sec. B.1 characterizes the training corpus by comparing y_o and y_r along length, verifier accuracy, and joint outcome on DeepScaleR (Qwen3-4B and Qwen3-8B, with-CoT and without-CoT subsets). Sec. B.2 drills into AMOBench at test time, decomposing Avg@16 and Pass@16 by base-difficulty bucket to localize where SRD’s gains arise. Sec. B.3 ablates Forward-KL, Reverse-KL, and SRD under *fail*, *succ*, and *fail*→*succ* corpus filters on Qwen3-8B.

B.1 Training-Trajectory Analysis: y_o vs. y_r

Here we inspect the training data on which SRD itself is trained, i.e., the raw rollout $y_o \sim \pi_\theta(\cdot | x)$ and the refined trajectory $y_r \sim \pi_\theta(\cdot | x, y_o, y^*)$ on DeepScaleR. The reference y^* supplied with each problem comes in two qualities: a small subset ($n=4,419$) carries a usable reference chain-of-thought, while the rest ($n=35,826$) carries only a short answer-style reference. We split the analysis along this axis to show that SRD’s behavior is consistent across both regimes, reporting Qwen3-4B-Instruct-2507 in Fig. 5 and Qwen3-8B in Fig. 6, with the with-CoT subset on the top row of each figure and the without-CoT subset on the bottom row. We report numbers as 4B / 8B and as with-CoT / without-CoT when the two subsets diverge.

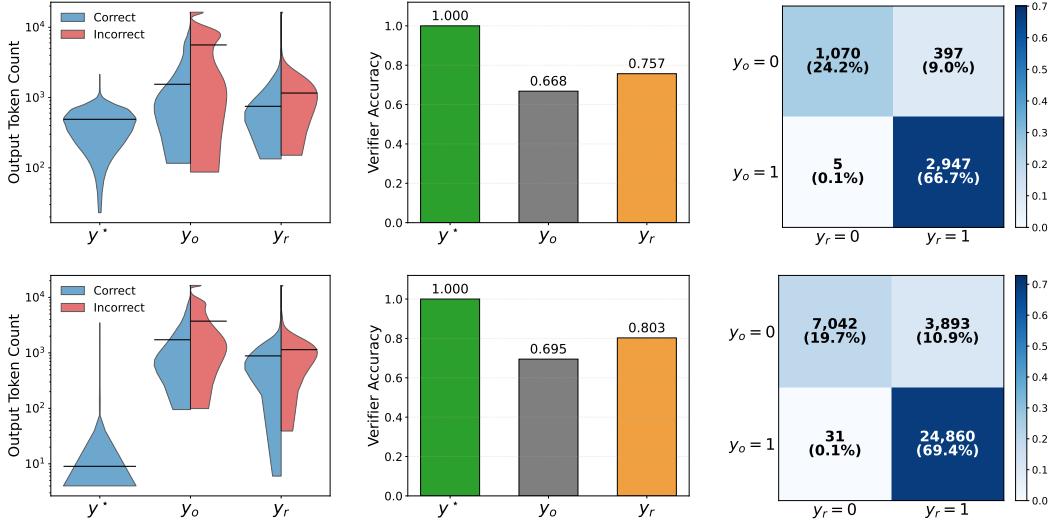


Figure 5: Training-trajectory analysis on Qwen3-4B-Instruct-2507. **Top row:** with-CoT subset ($n=4,419$). **Bottom row:** Without-CoT subset ($n=35,826$). **Left:** Length distribution of y^* , y_o , y_r . With y_o and y_r split into correct (left) and incorrect (right) halves. **Middle:** Verifier accuracy of the three trajectories. **Right:** Joint outcome of y_o and y_r (2×2 confusion); fail→pass cells reflect prefix-failure recovery, pass→fail cells quantify the price.

Refinement compresses the trajectory. The left panels show that y_r moves substantially below y_o in both regimes. With CoT, y^* has median ~ 0.49 K tokens and y_r collapses to 0.85 K / 0.88 K from y_o ’s 2.2 K / 7.7 K (4B / 8B); without CoT, y^* shrinks to ~ 9 tokens (answer-only) and y_r lands at 0.93 K / 0.83 K from y_o ’s 2.1 K / 7.5 K. The compression factor on 8B is similar in both regimes ($\sim 9\times$), confirming that conditioning on y^* pulls the student toward more concise derivations even when the reference is just a short answer string.

Refinement raises verifier accuracy. The middle panels report verifier accuracy. With CoT, y_o passes on 66.8% / 65.8% while y_r passes on 75.7% / 81.4% (+8.9pp / +15.6pp). Without CoT, y_o passes on 69.5% / 69.8% and y_r on 80.3% / 79.8% (+10.8pp / +10.0pp). SRD’s training data therefore contains a substantially higher fraction of correct trajectories than what dense-KL baselines train on across both regimes.

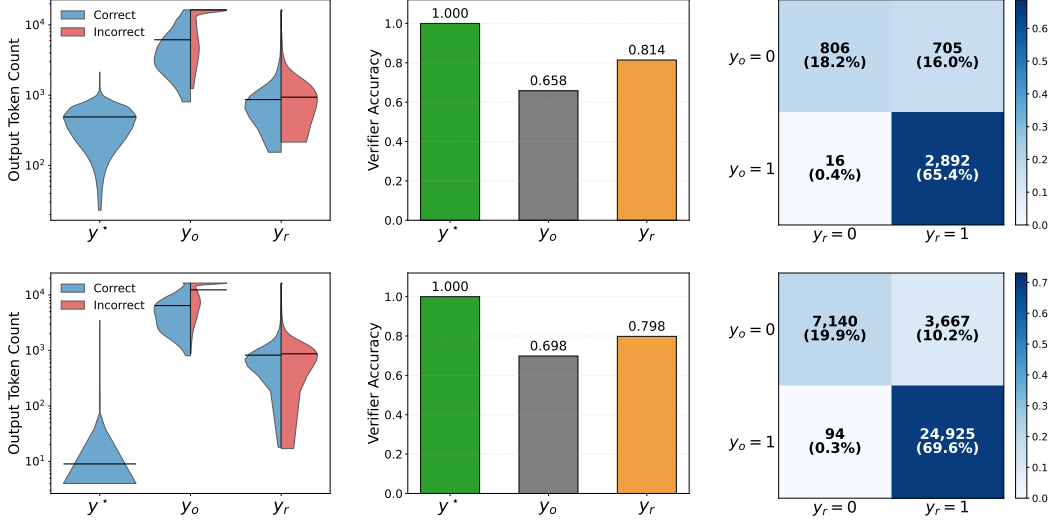


Figure 6: Training-trajectory analysis on Qwen3-8B. **Top row:** with-CoT subset ($n=4,419$). **Bottom row:** Without-CoT subset ($n=35,826$). **Left:** Length distribution of y^* , y_o , y_r , with y_o and y_r split into correct (left) and incorrect (right) halves. **Middle:** Verifier accuracy of the three trajectories. **Right:** Joint outcome of y_o and y_r (2×2 confusion); fail→pass cells reflect prefix-failure recovery, pass→fail cells quantify the price.

Refinement is monotonically corrective. The right panels decompose the accuracy gap by joint outcome. The fail→pass to pass→fail asymmetry is large in every cell ($\sim 80\times$ on 4B-with-CoT, $\sim 44\times$ on 8B-with-CoT, with comparable ratios on the without-CoT subset), consistent with the prefix-failure mechanism in Sec. 4, i.e., reference-guided refinement primarily corrects dead-end prefixes rather than disturbing already-correct ones. Refinement is not a panacea, around a quarter to a third of y_o failures still survive after refinement and a sub-1% pass→fail leakage remains in every setting, but the 8B runs consistently lift both the recovery rate and absolute accuracy, suggesting the residual fail→fail mass shrinks as the student grows more capable of producing y_o and consuming y^* .

B.2 Test Rollout Analysis

Setup. We take the same Qwen3-8B checkpoints used for Tabs. 1 and 2 and analyze their test-time rollouts on AMOBench (An et al., 2025), the most distillation-sensitive of our five benchmarks (largest absolute Pass@16 gain in Tab. 2). For each of the 39 AMOBench questions we draw $K=16$ independent completions per method, with the same generation parameters as in Sec. 6.1 (temperature 0.6, top- $p=0.95$, 38,912-token response budget). Each completion is then (I) tokenized with the Qwen3-8B tokenizer to obtain its output length, and (II) graded by the AMOBench rule-based verifier. Among the four dense-KL baselines we compare against +Forward KL (no clip), the strongest exploration-side baseline on AMOBench Pass@16 in Tab. 2.

Base-difficulty buckets. To isolate where each method helps, we partition the 39 AMOBench questions by the base model’s per-question pass count $b_q := \sum_{i=1}^{16} \text{Verify}(y_q^{(i, \text{base})}, y_q^*) \in \{0, \dots, 16\}$ (the number of base-model rollouts that pass the verifier). This yields three difficulty buckets:

- **B0** ($n = 23$): questions on which the base model fails all 16 attempts. By construction, these are *unreachable* for the base policy at $K=16$ sampling, so any positive Pass@16 in this bucket reflects support expansion rather than sharpening.
- **B1–8** ($n = 12$): medium-difficulty questions the base model solves between 1 and 8 times out of 16, where Avg@16 has the most headroom and sharpening is meaningful.
- **B9–16** ($n = 4$): easy questions the base model already solves at least 9 times out of 16, near the saturation ceiling for both Avg@16 and Pass@16.

Bucket sizes are determined by the base model and held fixed when scoring +Forward KL and SRD, so the same question belongs to the same bucket across all three methods.

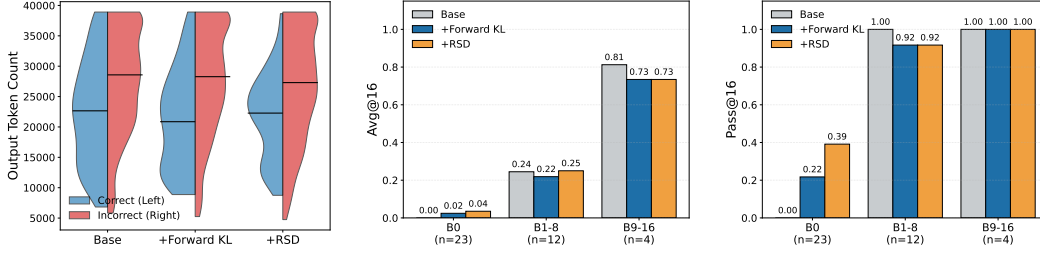


Figure 7: Test-rollout analysis on AMOBench (Qwen3-8B, $K=16$ samples per question). **Left:** Per-rollout response-length distribution split by correctness (correct on the left half, incorrect on the right half of each violin). **Middle / Right:** Avg@16 and Pass@16 stratified by base-model difficulty bucket, where B0 groups the 23 questions the base model fails on all 16 attempts, B1–8 the 12 questions solved between 1 and 8 times, and B9–16 the 4 questions solved at least 9 times.

SRD finds shorter solution paths. The left panel of Fig. 7 shows that SRD’s correct-rollout distribution is bimodal, with a pronounced low-length mode (around 10^4 tokens) absent in both Base and +Forward KL, indicating that SRD finds noticeably shorter reasoning chains on the problems it can solve. Incorrect distributions are similar across methods (bunched against the generation cap), so the accuracy gains below come without longer reasoning, i.e., the (accuracy, compute) trade-off moves in the right direction.

Avg@16 by bucket: sharpening on medium difficulty. The middle panel localizes the per-sample improvement. On B1–8, SRD lifts Avg@16 to 0.25, above both Base (0.24) and +Forward KL (0.22). The fact that +Forward KL *regresses* the base model on the same bucket indicates that this sharpening is SRD-specific rather than a generic property of dense-KL distillation. On B0, the absolute Avg@16 is small (0.04 for SRD vs 0.02 for +Forward KL), but every positive sample on B0 is a trajectory the base policy never produces under $K=16$, so SRD’s per-sample exploration rate on these questions is roughly twice that of +Forward KL.

Pass@16 by bucket: frontier expansion on hard questions. The right panel makes the exploration story explicit. On B0, the 23 AMOBench questions where the base model fails on all 16 attempts, SRD achieves Pass@16 = 0.39, nearly doubling the strongest baseline (+Forward KL at 0.22). Because B0 questions are by construction unreachable for the base policy at $K=16$, any positive Pass@16 here is direct evidence that SRD expands the reachable support of the base policy rather than only sharpening the existing distribution. The mild regression on B1–8 Pass@16 (–1/12 questions) is overwhelmed by the +9/23 gain on B0, leaving the dataset-level Pass@16 in Tab. 2 net positive.

B.3 Ablation: Trajectory-Subset for OPSD on Qwen3-8B

Each table fixes one algorithm and varies the training corpus by filtering on the outcome of y_o and y_r . The three subset filters partition (x, y_o, y_r) tuples along the student’s verifier outcome on y_o . *fail* keeps tuples where y_o is incorrect ($n=12,318$), *succ* keeps those where y_o is correct ($n=27,927$), and *fail*→*succ* keeps the intersection where y_o is incorrect and y_r is correct ($n=4,372$). The full corpus is the union *fail* $\cup$ *succ* over $n=40,245$ DeepScaleR problems and reproduces the no-subset main-table result for each algorithm. Subset rows below the rule report the change relative to the algorithm’s no-subset row (e.g., Forward-KL subset rows compare against +Forward KL, not Base).

Both fail and succ halves are necessary for coverage. Across Tabs. 3 to 5, no subset filter is uniformly better than the no-subset corpus for any algorithm. Per-column wins under a filter (e.g., +Forward KL (fail) BeyondAIME Avg@16, +Reverse KL (fail) BeyondAIME Pass@16) are paid for by regressions elsewhere in the same row. Both halves contribute training coverage the per-token KL exploits, so the full corpus is the right default.

Table 3: Forward-KL ablation by training subset (Qwen3-8B). Red cells mark drops below that reference; bold marks the column maximum.

Method	Traj.	Avg@16 (%)					Pass@16 (%)				
		AIME24	AIME25	HMMT25	BeyondAIME	AMOBench	AIME24	AIME25	HMMT25	BeyondAIME	AMOBench
Base	–	76.5	66.7	41.5	41.6	15.9	90.0	83.3	66.7	66.0	41.0
+SRD (ours)	y_r	76.5	69.2	44.5	42.8	17.3	90.0	86.7	73.3	68.0	61.5
+Forward KL	y_o	74.8	65.4	40.3	39.6	15.7	86.7	82.7	73.3	68.0	51.3
+Forward KL (fail)	y_o	74.9 (+0.1)	68.1 (+2.7)	40.1 (-0.2)	42.8 (+3.2)	16.3 (+0.6)	85.3 (-1.4)	86.7 (+4.0)	70.0 (-3.3)	67.0 (-1.0)	46.2 (-5.1)
+Forward KL (succ)	y_o	75.0 (+0.2)	67.7 (+2.3)	40.1 (-0.2)	40.1 (+0.5)	15.8 (+0.1)	86.7 (+0.0)	83.3 (+0.6)	73.3 (+0.0)	64.6 (-3.4)	46.2 (-5.1)

Table 4: Reverse-KL ablation by training subset (Qwen3-8B). Red cells mark drops below that reference; bold marks the column maximum.

Method	Traj.	Avg@16 (%)					Pass@16 (%)				
		AIME24	AIME25	HMMT25	BeyondAIME	AMOBench	AIME24	AIME25	HMMT25	BeyondAIME	AMOBench
Base	–	76.5	66.7	41.5	41.6	15.9	90.0	83.3	66.7	66.0	41.0
+SRD (ours)	y_r	76.5	69.2	44.5	42.8	17.3	90.0	86.7	73.3	68.0	61.5
+Reverse KL	y_o	75.4	68.2	44.3	40.8	16.8	86.7	86.7	73.3	64.0	51.3
+Reverse KL (fail)	y_o	75.4 (+0.0)	69.2 (+1.0)	44.0 (-0.3)	42.4 (+1.6)	16.7 (-0.1)	83.3 (-3.4)	86.7 (+0.0)	66.7 (-6.6)	70.0 (+6.0)	53.8 (+2.5)
+Reverse KL (succ)	y_o	75.4 (+0.0)	69.2 (+1.0)	44.2 (-0.1)	42.3 (+1.5)	16.2 (-0.6)	83.3 (-3.4)	86.7 (+0.0)	70.0 (-3.3)	68.0 (+4.0)	46.2 (-5.1)

Table 5: SRD ablation by training subset (Qwen3-8B). Red cells mark drops below that reference; bold marks the column maximum.

Method	Traj.	Avg@16 (%)					Pass@16 (%)				
		AIME24	AIME25	HMMT25	BeyondAIME	AMOBench	AIME24	AIME25	HMMT25	BeyondAIME	AMOBench
Base	–	76.5	66.7	41.5	41.6	15.9	90.0	83.3	66.7	66.0	41.0
+SRD (ours)	y_r	76.5	69.2	44.5	42.8	17.3	90.0	86.7	73.3	68.0	61.5
+SRD (fail $\rightarrow \cdot$)	y_r	75.4 (-1.1)	69.2 (+0.0)	41.9 (-2.6)	43.3 (+0.5)	17.3 (+0.0)	86.7 (-3.3)	83.3 (-3.4)	66.7 (-6.6)	64.0 (-4.0)	51.3 (-10.2)
+SRD (succ $\rightarrow \cdot$)	y_r	75.4 (-1.1)	70.4 (+1.2)	43.5 (-1.0)	43.3 (+0.5)	15.9 (-1.4)	86.7 (-3.3)	86.9 (+0.2)	70.0 (-3.3)	65.0 (-3.0)	46.2 (-15.3)
+SRD (fail $\rightarrow$ succ)	y_r	75.4 (-1.1)	69.0 (-0.2)	42.9 (-1.6)	43.1 (+0.3)	16.8 (-0.5)	86.7 (-3.3)	83.3 (-3.4)	70.0 (-3.3)	67.0 (-1.0)	51.3 (-10.2)

Filtering is not an optimization of Eq. (6). Subset filtering changes which trajectories enter $\mathcal{D}$ but not the loss itself, so neither *succ*-only nor *fail*-only is an optimization of Eq. (6). Each filter also drops a complementary signal. *succ*-only loses the hard problems on which the student fails unaided, where the teacher provides extra signal that raises the probability of reaching previously unreachable solutions. *fail*-only loses the alternative-path signal on the easy half, where the teacher can offer stronger or shorter derivations than the student would produce on its own.

Forward KL is the most data-sensitive. Vanilla +Forward KL regresses Base on four of five Avg@16 benchmarks (AIME24 -1.7 , AIME25 -1.3 , HMMT25 -1.2 , BeyondAIME -2.0). Filtering recovers AIME25 (fail $+2.7$, succ $+2.3$) and BeyondAIME (fail $+3.2$), but AIME24 and HMMT25 remain below Base under any filter, and Pass@16 is mostly traded down (AMOBench -5.1 under both filters). This column-specific trade is consistent with the mass-spread character of the mode-covering forward KL, which cannot ignore points in the corpus and therefore inherits both the support coverage and the prefix-failure pressure of whichever subset it is fed.

Reverse KL is almost flat under filtering. Both subset rows sit within ± 1.0 of the no-filter Avg@16 and trade Pass mildly (BeyondAIME up, AMOBench down). The mode-seeking reverse KL already discounts low-probability regions, so removing the fail or succ half does not change the optimization much. This is a stability story, not a quality story, and the no-subset Reverse KL is therefore not noticeably improved by curation.

SRD is hurt by filtering, especially on Pass@16. Every subset row drops Pass@16 on every benchmark, with double-digit losses on AMOBench (-10 to -15). Avg@16 deltas are small and mixed. The fail $\rightarrow$ succ row ($n=4,372$, the "ideal" subset where refinement fixed errors) does not outperform full corpus, exhibiting the same Pass losses and no Avg gains worth the data cut. SRD's gain comes from the breadth of the refined corpus rather than from any privileged subset, supporting the choice to train on y_r over the unfiltered corpus by default.

C Experiment Details

This appendix collects the training data and trajectory construction (Sec. C.1), hardware and wall-clock budget (Sec. C.3), the refinement prompt template (Sec. C.4), the training-metric extraction used in Fig. 2 (Sec. C.5), models and distillation setup (Sec. C.6), method-specific and common optimization hyperparameters (Secs. C.7 and C.8), and the evaluation protocol (Sec. C.2) used throughout Sec. 6.

C.1 Training Data and Trajectory Construction

We train on DeepScaleR (Luo et al., 2025), a math problem corpus with verified answers, approximately 40K problems each with a reference solution y^* . The release is de-duplicated against AIME problems, ensuring no overlap with AIME24 in our evaluation; AIME25, HMMT25, BeyondAIME, and AMOBench post-date the corpus and are by construction non-overlapping. For every problem x we run a two-stage offline pipeline before any KL update.

Stage 1 (raw rollout y_o). The current student samples one rollout per prompt with vLLM (Kwon et al., 2023) using temperature 0.6, top- $p = 0.95$, top- $k = 20$, prompt budget 4096 tokens, and response budget 16,384 tokens. Each rollout is scored by a rule-based verifier (DeepSeek-AI, 2025) against y^* to populate a binary outcome label used only for the D bucketing in Fig. 2 (training itself does not use this label).

Stage 2 (refined trajectory y_r , SRD only). Conditioning on the same problem, the raw rollout, and the reference, the student samples $y_r \sim \pi_\theta(\cdot \mid x, y_o, y^*)$ via the prompt template in Sec. C.4. Sampling parameters match Stage 1 except the prompt budget grows to 24,576 tokens to accommodate the concatenated initial-solution and reference fields. Stage 2 runs over all problems, regardless of whether y_o passed Stage 1. The dense-KL baselines train on the Stage 1 parquet and skip Stage 2.

C.2 Evaluation Protocol

Evaluation is run on the merged-weights checkpoint after Stage 2 training. We sample 16 completions per problem with vLLM at temperature 0.6, top- $p = 0.95$, prompt budget 4,096 tokens, and response budget 38,912 tokens (chosen to avoid truncation on long thinking traces). Each completion is scored by the same rule-based verifier used in Stage 1, with answer extraction from the final `\boxed{...}` block.

Metrics. For each test question x_q we draw $K=16$ independent completions $y_q^{(i)} \sim \pi_\theta(\cdot \mid x_q)$ and score them with the verifier $\text{Verify}(\cdot, \cdot)$ from Eq. (5). Over N test questions,

$$\text{Avg@}K = \frac{1}{N} \sum_{q=1}^N \frac{1}{K} \sum_{i=1}^K \text{Verify}(y_q^{(i)}, y_q^*), \quad \text{Pass@}K = \frac{1}{N} \sum_{q=1}^N \max_{1 \leq i \leq K} \text{Verify}(y_q^{(i)}, y_q^*).$$

Avg@16 tracks sharpening of the modal answer; Pass@16 tracks the breadth of the student’s reasoning support.

Benchmarks. AIME24 and AIME25 (American Invitational Mathematics Examination, 30 problems each) (Mathematical Association of America, 2024, 2025); HMMT25 (Harvard-MIT Mathematics Tournament February 2025, 30 problems) (HMMT, 2025); BeyondAIME (a curated set of competition problems harder than AIME) (Seed, 2025); and AMOBench (Asian Mathematical Olympiad benchmark) (An et al., 2025). AMOBench consists of 50 hand-designed problems, 39 with parser-based grading and 11 with descriptive (free-form) answers; we evaluate on the 39 parser-graded problems only, so they share the same rule-based grader as the other four benchmarks. All numbers reported under *AMOBench* therefore refer to this 39-problem subset.

C.3 Hardware and Compute

All runs use a single node of $8 \times$ H100 80GB GPUs with FSDP2 sharding via verl (Sheng et al., 2024). Each row of Tabs. 1 and 2 corresponds to one offline pipeline run, comprising Stage 1 generation (raw

rollout y_o), optional Stage 2 generation (refined trajectory y_r , SRD only), and one training epoch over the resulting parquet.

Training-time trade-off. A non-obvious consequence of the trajectory compression in [Sec. B.1](#) is a substantial training-time saving. Dense-KL baselines train on y_o (median $\sim 7.7\text{K}$ tokens on Qwen3-8B); SRD trains on y_r (median $\sim 0.88\text{K}$). Per-step compute scales with two length-dependent quantities, (I) transformer forward/backward (attention quadratic, MLP linear in length), and (II) the full-vocabulary KL head, which materializes $|\mathcal{V}| \approx 152\text{K}$ logits per kept token. The $\sim 9\times$ shorter y_r therefore translates almost directly into faster training even at matched effective batch.

Measured wall-clock. [Tab. 6](#) reports per-method end-to-end wall-clock, organized by the stages each method actually runs. Forward KL and Reverse KL train on y_o , so they only pay the Stage 1 sampling cost before training. SRD additionally runs Stage 2 (sampling y_r via the refinement prompt of [Sec. C.4](#)), but its training step operates on the much shorter y_r and finishes in roughly 40% of the time, e.g., 2h10m on Qwen3-8B vs 5h30m for Reverse KL on y_o . On Qwen3-8B the training saving more than cancels the Stage 2 cost, so SRD runs end-to-end in 9h20m, 20–30 minutes faster than the dense-KL baselines (9h40m / 9h50m) despite paying for an additional sampling stage. On Qwen3-4B the absolute training time is small to begin with, so SRD pays a residual $\sim 1\text{h}10\text{m}$ overhead, but the absolute scale is short.

Table 6: End-to-end wall-clock per method (h:mm, rounded to 10 minutes) on a single $8\times \text{H100}$ 80GB node. Forward KL and Reverse KL train on y_o and skip Stage 2; SRD additionally runs Stage 2 but trains on the much shorter y_r . Evaluation ([Tab. 7](#)) is shared across methods and excluded from the total.

Backbone	Method	Stage 1 (y_o)	Stage 2 (y_r)	Training	Total
Qwen3-4B-Instruct-2507	Forward KL	2:10	—	2:10	4:20
	Reverse KL	2:10	—	2:10	4:20
	SRD (ours)	2:10	2:00	1:20	5:30
Qwen3-8B	Forward KL	4:20	—	5:20	9:40
	Reverse KL	4:20	—	5:30	9:50
	SRD (ours)	4:20	2:50	2:10	9:20

We therefore view the Stage 2 sampling pass not as a strict additional cost but as a budget redistribution from training to inference, with a favorable signal-quality trade ([Sec. B.1](#)).

Evaluation wall-clock. [Tab. 7](#) reports per-benchmark generation time for the $K=16$ rollouts per question used to compute Avg@16 and Pass@16. The same evaluation budget is paid by every method (Base, dense-KL baselines, SRD), so it does not affect the relative comparison in [Tab. 6](#).

Table 7: Evaluation wall-clock per benchmark (h:mm, rounded to 10 minutes) on a single $8\times \text{H100}$ 80GB node. *Questions* is the original test-set size; *Rollouts* = $N \times K$ with $K=16$ samples per question.

Benchmark	Questions	Rollouts	4B time	8B time
AIME24	30	480	0:20	0:30
AIME25	30	480	0:20	0:40
HMMT25	30	480	0:20	0:30
BeyondAIME	100	1,600	0:30	1:20
AMOBench	39	624	0:20	1:00
Total	229	3,664	$\sim 2:00$	$\sim 4:00$

C.4 Refinement Prompt

For each problem x , raw rollout y_o , and reference solution y^* , we use the following instruction to sample the refined trajectory y_r :

SRD Prompt

Your task is to rewrite your mathematical solution using the reference solution as guidance.

Problem:

{PROBLEM}

Reference Solution:

{EXPERT SOLUTION}

Your Initial Solution:

{INITIAL RESPONSE}

Instructions:

1. Review the reference solution to understand the target reasoning and method
2. Rewrite your solution so it is consistent with the reference solution
3. Keep useful parts of original solutions' structure and style when appropriate
4. Output ONLY the rewritten solution

C.5 Training Metrics for Fig. 2

We log three diagnostics during OPSD training, all computed on student rollouts $y_o \sim \pi_\theta(\cdot | x)$ over response tokens (with mask $m_{i,t} \in \{0, 1\}$). All three are first accumulated as numerator/denominator within each step, then divided, so the reported value is a token-weighted mean.

Per-token KL by rollout outcome. Each prompt x_i carries a binary stage-1 outcome label b_i , set to 1 (*correct*) if the base model's stage-1 rollout passes the verifier and to 0 (*incorrect*) otherwise. The per-token KL between teacher and student is averaged separately within each bucket,

$$D^{\text{correct}} = \frac{\sum_{i: b_i=1} \sum_t D_{\text{KL},i,t} m_{i,t}}{\sum_{i: b_i=1} \sum_t m_{i,t}}, \quad D^{\text{incorrect}} = \frac{\sum_{i: b_i=0} \sum_t D_{\text{KL},i,t} m_{i,t}}{\sum_{i: b_i=0} \sum_t m_{i,t}},$$

where $D_{\text{KL},i,t}$ is the per-token KL term in Eq. (1). The values are directly comparable to the global D since both use a token-weighted denominator.

Epistemic-token mass. Let $\mathcal{E} \subset \mathcal{V}$ be the set of *epistemic onset* tokens. We construct $\mathcal{E}$ from 16 phrases,

Wait, Actually, However, Alternatively, Oops, Wrong, Error, Incorrect, Correction,
Sorry, Hmm, Oh, Hold, Pause, Uh, Um,

by tokenising each phrase under the student tokenizer in two variants (bare string and leading-space) and collecting the first sub-word id, then deduplicating. At each response position (i, t) we measure how much teacher mass falls on $\mathcal{E}$ before any KL temperature scaling,

$$\text{mass}_{i,t} = \sum_{v \in \mathcal{E}} \pi_T(v | x_i, y_i^*, y_{<t}) = \sum_{v \in \mathcal{E}} \text{softmax}(\ell_{i,t}^T)_v,$$

and report the token-weighted mean $(\sum_{i,t} \text{mass}_{i,t} m_{i,t}) / (\sum_{i,t} m_{i,t})$. The metric is computed only on the full-vocabulary path (i.e., when teacher logits are materialised) and is independent of the KL-loss temperature.

Teacher-student perplexity gap. Both perplexities are token-weighted under teacher-forced decoding on the same response mask,

$$\text{PPL}_S = \exp\left(\frac{\sum_{i,t} -\log \pi_S(y_{i,t} | y_{<t}) m_{i,t}}{\sum_{i,t} m_{i,t}}\right), \quad \text{PPL}_T = \exp\left(\frac{\sum_{i,t} -\log \pi_T(y_{i,t} | y_{<t}) m_{i,t}}{\sum_{i,t} m_{i,t}}\right).$$

Since both share the same mask and use $T=1$ log-probabilities, the gap $\text{PPL}_S - \text{PPL}_T$ is directly comparable across runs.

C.6 Models and Distillation Setup

Base models are Qwen3-4B-Instruct-2507 and Qwen3-8B with thinking enabled (Yang et al., 2025). Throughout, the same backbone embodies teacher and student via the privileged-context substitution $\pi_T(\cdot \mid x, y_{<t}) = \pi_\theta(\cdot \mid x, y^*, y_{<t})$ (Eq. (1)); LoRA adapters update only the student branch, and the stop-gradient operator on the teacher branch is implemented by detaching π_T ’s logits before the KL is computed. We apply LoRA of rank $r=64$, scaling $\alpha=128$, dropout 0.05, on all attention and MLP linear layers (q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj). After training we merge the adapters into the base weights for evaluation.

C.7 Method-Specific Hyperparameters

Tab. 8 lists the per-method settings that distinguish each row of Tabs. 1 and 2. All five methods use the full-vocabulary KL estimator (loss head materialized over the full Qwen3 vocabulary, $|\mathcal{V}| \approx 152\text{K}$), AdamW, bfloat16, gradient checkpointing, and a single training epoch. Effective batch size is computed as (per-GPU batch) $\times$ (GPUs) $\times$ (gradient accumulation), assuming 8 GPUs per node.

Table 8: Per-method training settings. *Forward KL w/ Clip* reproduces the OPSD recipe of Zhao et al. (2026); *Reverse KL w/ Top-K* reproduces the truncated-support objective of Fu et al. (2026).

Method	Traj.	KL dir.	Clip c	Top- K	Temp. T	LR	Eff. batch	Max len.
Forward KL	y_o	forward	0	—	1.0	5×10^{-6}	32	18432
Forward KL w/ Clip (OPSD)	y_o	forward	0.06	—	1.1	5×10^{-6}	32	18432
Reverse KL	y_o	reverse	0	—	1.0	2×10^{-5}	256	18432
Reverse KL w/ Top- K	y_o	reverse	0	32	1.0	2×10^{-6}	128	18432
SRD (ours)	y_r	forward	0	—	1.0	2×10^{-5}	512	20480

C.8 Common Optimization Hyperparameters

Settings shared across all methods are listed in Tab. 9. Reverse KL w/ Top- K uses zero warmup and zero weight decay following Fu et al. (2026); all other methods use the values in the table.

Table 9: Common optimization hyperparameters.

Setting	Value
Optimizer	AdamW ($\beta_1=0.9$, $\beta_2=0.999$, $\epsilon=10^{-8}$)
Precision	bfloat16
Gradient checkpointing	enabled
Gradient clipping	1.0
LR schedule	linear warmup, cosine decay to $0.1 \times$ peak LR
Warmup ratio	0.1 (Reverse KL w/ Top- K uses 0)
Weight decay	0.0 (Reverse KL uses 0.005)
Epochs	1
Sequence packing	remove-padding via verl FSDP2
Sequence parallel	ulysses, size 1

Loss formulation. For each method, the per-token KL is computed in full vocabulary at temperature T as

$$D^{(T)}(p \parallel q) = \sum_{v \in \mathcal{V}} p_T(v) (\log p_T(v) - \log q_T(v)),$$

with $p_T(v) \propto \exp(\ell_v/T)$. The clipping baseline applies the OPSD per-token cap $\min(D_{\text{KL},t}, c)$ before averaging over the response mask. Top- K replaces $\mathcal{V}$ by the teacher’s top-32 support $\mathcal{S}_t$ and renormalizes both p_T and q_T to sum to 1 on $\mathcal{S}_t$ before evaluating D .

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [Yes]

Justification: The abstract and [Sec. 1](#) state our two contributions, identifying *prefix failure* as a structural limitation of dense-KL on-policy distillation ([Sec. 4](#)) and proposing SRD as a trajectory-level refinement ([Sec. 5](#)); both are validated empirically in [Sec. 6](#).

Guidelines:

- The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [Yes]

Justification: [Sec. 7](#) discusses two limitations: SRD adds one extra sampling pass per problem (offset by the $\sim 9\times$ length compression of y_r , see [Sec. C.3](#)), and SRD relies on the base model having sufficient capacity to produce a correct continuation under privileged context.

Guidelines:

- The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren’t acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [Yes]

Justification: The gradient derivation and supervision-pair-set analysis in [Sec. 4.2](#) state all assumptions inline; the full sequence-level reverse-KL policy-gradient derivation is given in [Sec. A](#).

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: [Sec. 6.1](#) and [Sec. C](#) fully specify the data (DeepScaleR), trajectory-construction pipeline (Stage 1 / Stage 2 sampling), refinement prompt ([Sec. C.4](#)), models (Qwen3-4B-Instruct-2507 and Qwen3-8B), LoRA configuration, optimizer hyperparameters, and evaluation protocol ([Sec. C.2](#)).

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).

- (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [No]

Justification: All datasets used (DeepScaleR, AIME24/25, HMMT25, BeyondAIME, AMOBench) and base models (Qwen3-4B-Instruct-2507, Qwen3-8B) are publicly available and cited in Sec. 6.1. Code is currently under organization and will be released upon publication; the algorithm and hyperparameters are fully specified in Sec. C to enable reproduction.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: Sec. 6.1 states the headline setup, and all training details are shown in Sec. C.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [No]

Justification: We report Avg@16 and Pass@16 with $K=16$ independent completions per test question (Sec. 6.1), which provides per-question averaging, but we do not run multi-seed training to compute error bars due to the wall-clock cost of full training runs (Sec. C.3).

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: Sec. C.3 specifies the hardware (single $8 \times$ H100 80GB node with FSDP2 via verl) and reports per-method end-to-end wall-clock for both Qwen3-4B and Qwen3-8B in Tab. 6, plus per-benchmark evaluation wall-clock in Tab. 7.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines?>

Answer: [Yes]

Justification: The research uses publicly released models and benchmarks for mathematical reasoning, involves no human subjects or personal data, and conforms to the NeurIPS Code of Ethics.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [N/A]

Justification: The paper proposes a foundational training technique for self-distillation on verifiable-reward tasks; it is not tied to a particular deployment or application and does not introduce a direct path to negative societal use beyond the general improvements in language-model post-training.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: We do not release new pre-trained models or scraped datasets; the paper only fine-tunes publicly released Qwen3 backbones on the publicly released DeepScaleR corpus.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: All used assets are cited at the point of use: Qwen3 base models (Yang et al., 2025), DeepScaleR training corpus (Luo et al., 2025), and the five evaluation benchmarks (Sec. C.2). We use them under their public terms and do not redistribute them.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, paperswithcode.com/datasets has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [N/A]

Justification: The paper does not release new datasets or model checkpoints at submission. The proposed method is fully documented in Sec. 5 and Sec. C; code release is planned upon publication.

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A]

Justification: The paper involves no crowdsourcing or research with human subjects.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

1007 **15. Institutional review board (IRB) approvals or equivalent for research with human**
 1008 **subjects**

1009 Question: Does the paper describe potential risks incurred by study participants, whether
 1010 such risks were disclosed to the subjects, and whether Institutional Review Board (IRB)
 1011 approvals (or an equivalent approval/review based on the requirements of your country or
 1012 institution) were obtained?

1013 Answer: [N/A]

1014 Justification: The paper involves no human subjects, so no IRB approval is required.

1015 Guidelines:

- 1016 • The answer [N/A] means that the paper does not involve crowdsourcing nor research
 1017 with human subjects.
- 1018 • Depending on the country in which research is conducted, IRB approval (or equivalent)
 1019 may be required for any human subjects research. If you obtained IRB approval, you
 1020 should clearly state this in the paper.
- 1021 • We recognize that the procedures for this may vary significantly between institutions
 1022 and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the
 1023 guidelines for their institution.
- 1024 • For initial submissions, do not include any information that would break anonymity (if
 1025 applicable), such as the institution conducting the review.

1026 **16. Declaration of LLM usage**

1027 Question: Does the paper describe the usage of LLMs if it is an important, original, or
 1028 non-standard component of the core methods in this research? Note that if the LLM is used
 1029 only for writing, editing, or formatting purposes and does *not* impact the core methodology,
 1030 scientific rigor, or originality of the research, declaration is not required.

1031 Answer: [N/A]

1032 Justification: LLMs are the object of study (the student/teacher being distilled, Qwen3-4B
 1033 and Qwen3-8B), not an external non-standard component of the methodology; their use is
 1034 fully described in [Sec. 6.1](#) and [Sec. C](#).

1035 Guidelines:

- 1036 • The answer [N/A] means that the core method development in this research does not
 1037 involve LLMs as any important, original, or non-standard components.
- 1038 • Please refer to our LLM policy in the NeurIPS handbook for what should or should not
 1039 be described.